Collibra MCP Server Integration with Tavro

1. Objective

The objective of this implementation is to integrate the Collibra MCP (Model Context Protocol) Server with the Tavro application so that Tavro can interact with Collibra metadata such as communities, domains, assets, etc through MCP tools.

2. Prerequisites

Before starting the integration, the following prerequisites are required:

2.1 Collibra Environment

A running Collibra instance with:

- Communities configured
- Domains configured
- Required assets available
- API access enabled
- User credentials with appropriate permissions

2.2 Tavro Application

- Tavro source code downloaded from the open-source repository
- Docker and Docker Compose installed on your system
- Git installed for version control
- Basic understanding of Docker container networking

2.3 System Requirements

- Disk Space: Approximately 10 GB or more for Docker images and containers
- RAM: Minimum 8GB (16GB recommended)
- Network: Port availability (8080 for Collibra chip, 9001 for MCP server, 9000 for Tavro API)

3. Integration Approach

The integration was done by running the Collibra MCP Server as a sidecar service and exposing selected Collibra MCP tools through Tavro's existing MCP server.

Instead of directly embedding Collibra logic inside Tavro, Tavro communicates with the Collibra MCP Server using JSON-RPC MCP calls. Tavro acts as a bridge between the application UI/assistant and the Collibra MCP tools.

4. Integration Steps

4.1 Clone the Tavro Repository

Clone the Tavro source code from:
git clone https://github.com/TavroOrg/tavro.git
cd tavro

4.2 Get the Collibra Chip Binary

Download the Collibra chip binary for Linux AMD64 from the Collibra open-source chip repository:
Github Repo Link: https://github.com/collibra/chip
Download the latest chip-linux-amd64 binary from the Releases page.

4.3 Place the Binary in the Tavro Repository

1. After cloning Tavro, create the directory and place the binary:
   third_party/collibra/
2. Copy the downloaded CHIP binary into this directory and rename it to "chip-linux-amd64"
3. The resulting directory structure should be:
   third_party/
   └── collibra/
   └── chip-linux-amd64

5. Collibra MCP Server Setup

Create a new file called Dockerfile.collibra-mcp at the root of the Tavro repository.
File: Dockerfile.collibra-mcp

Code:

```
# syntax=docker/dockerfile:1.6
FROM debian:bookworm-slim

ARG CHIP_BINARY=third_party/collibra/chip-linux-amd64

RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt/lists,sharing=locked \
    apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY ${CHIP_BINARY} /app/chip

RUN chmod +x /app/chip

EXPOSE 8080

CMD ["/app/chip"]
```

This Dockerfile copies the Collibra chip binary into a lightweight Debian container, makes it executable, exposes port `8080`, and starts the chip process.

The chip binary path is configurable using:

```env
COLLIBRA_CHIP_BINARY_PATH=third_party/collibra/chip-linux-amd64
```

6. Docker Compose Integration

A new service was added in docker-compose.yml for the Collibra MCP Server.

Service name: collibra-mcp-server

This service builds from Dockerfile.collibra-mcp and runs the Collibra chip as a sidecar.

Open docker-compose.yml and add the following new service at the end of the services block:

```
  collibra-mcp-server:
    build:
      context: .
      dockerfile: Dockerfile.collibra-mcp
      args:
        CHIP_BINARY:
${COLLIBRA_CHIP_BINARY_PATH:-third_party/collibra/chip-linux-amd64}
    container_name: collibra-mcp-server
    restart: unless-stopped
    network_mode: "service:risk-mcp-server"
    environment:
      COLLIBRA_MCP_API_URL: ${COLLIBRA_MCP_API_URL:-}
      COLLIBRA_MCP_API_USR: ${COLLIBRA_MCP_API_USR:-}
      COLLIBRA_MCP_API_PWD: ${COLLIBRA_MCP_API_PWD:-}
      COLLIBRA_MCP_MODE: ${COLLIBRA_MCP_MODE:-http}
      COLLIBRA_MCP_HTTP_PORT: ${COLLIBRA_MCP_HTTP_PORT:-8080}
      COLLIBRA_MCP_ENABLED_TOOLS:
${COLLIBRA_MCP_ENABLED_TOOLS:-discover_data_assets,get_asset_details,di
scover_business_glossary,create_asset}
    depends_on:
      risk-mcp-server:
        condition: service_started
```

Also find the existing risk-mcp-server service in docker-compose.yml and add this line to its environment block:
COLLIBRA_MCP_BASE_URL: ${COLLIBRA_MCP_BASE_URL:-}

This is required so Tavro's MCP server knows where to reach the Collibra chip sidecar.

The network_mode: "service:risk-mcp-server" means both containers share the same network
namespace. This is why Tavro can call the Collibra MCP Server at http://localhost:8080 without any port being exposed publicly.

7. Environment Configuration

These variables allow Tavro to connect to the Collibra MCP Server and allow the Collibra MCP Server to connect to the Collibra instance.

Open env_sample.txt and add the following block:

COLLIBRA_MCP_BASE_URL=http://localhost:8080

```
COLLIBRA_MCP_API_URL=https://your-collibra-instance.com/
COLLIBRA_MCP_API_USR=<your_collibra_username>
COLLIBRA_MCP_API_PWD=<your_collibra_password>
COLLIBRA_MCP_MODE=http
COLLIBRA_MCP_HTTP_PORT=8080
COLLIBRA_MCP_ENABLED_TOOLS=discover_data_assets,get_asset_details,discover_bu
siness_glossary,create_asset
COLLIBRA_MCP_TIMEOUT_SECONDS=45
COLLIBRA_CHIP_BINARY_PATH=third_party/collibra/chip-linux-amd64
```

Then copy env_sample.txt to .env and fill in your actual Collibra credentials:

Update .env with your values:

```
COLLIBRA_MCP_API_URL=https://your-collibra-instance.com/
COLLIBRA_MCP_API_USR=<your_collibra_username>
COLLIBRA_MCP_API_PWD=<your_collibra_password>
```

8. Update Tavro MCP Server

Open mcp_server/server.py and make the following changes.

8.1 Add Collibra Configuration Variables

After the existing environment variable reads at the top of the file, add:

```
COLLIBRA_MCP_BASE_URL = os.getenv("COLLIBRA_MCP_BASE_URL", "").strip()
COLLIBRA_MCP_API_USR = os.getenv("COLLIBRA_MCP_API_USR", "").strip()
COLLIBRA_MCP_API_PWD = os.getenv("COLLIBRA_MCP_API_PWD", "").strip()
COLLIBRA_MCP_TIMEOUT_SECONDS =
float(os.getenv("COLLIBRA_MCP_TIMEOUT_SECONDS", "45").strip() or "45")
```

8.2 Add Bridge Helper Functions

Add these functions before the tool definitions. They handle all JSON-RPC communication with the Collibra chip sidecar:

```
def _collibra_basic_auth() -> Optional[httpx.BasicAuth]:
    if not COLLIBRA_MCP_API_USR or not COLLIBRA_MCP_API_PWD:
        return None
    return httpx.BasicAuth(COLLIBRA_MCP_API_USR, COLLIBRA_MCP_API_PWD)


def _parse_mcp_http_response(body: str) -> Dict[str, Any]:
    if not body:
        raise ValueError("Empty MCP response")
    data_lines: List[str] = []
    for line in body.splitlines():
```

```
            stripped = line.strip()
            if stripped.startswith("data:"):
                data_lines.append(stripped[len("data:"):].strip())
        if data_lines:
            return json.loads("".join(data_lines))
        return json.loads(body)


    def _extract_text_from_content_blocks(content_blocks: Any) -> str:
        if not isinstance(content_blocks, list):
            return ""
        parts: List[str] = []
        for block in content_blocks:
            if isinstance(block, dict) and block.get("type") == "text" and
isinstance(block.get("text"), str):
                parts.append(block["text"])
        return "\n".join(part for part in parts if part).strip()


    def _normalize_collibra_tool_result(tool_name: str, payload: Dict[str, Any]) -> Dict[str,
Any]:
        result = payload.get("result") if isinstance(payload, dict) else None
        if not isinstance(result, dict):
            raise ValueError(f"Unexpected MCP payload for {tool_name}: missing result object")
        structured = result.get("structuredContent")
        text_content = _extract_text_from_content_blocks(result.get("content"))
        normalized: Dict[str, Any] = {
            "tool": tool_name,
            "structuredContent": structured,
            "content": text_content,
        }
        if structured is None and text_content:
            try:
                normalized["parsedContent"] = json.loads(text_content)
            except json.JSONDecodeError:
                pass
        return normalized


    async def _call_collibra_mcp_method(method: str, params: Optional[Dict[str, Any]] =
None) -> Dict[str, Any]:
        if not COLLIBRA_MCP_BASE_URL:
            raise ValueError("COLLIBRA_MCP_BASE_URL is not configured")
        payload = {
            "jsonrpc": "2.0",
            "id": str(uuid.uuid4()),
            "method": method,
            "params": params or {},
```

```
        }
        headers = {
            "Content-Type": "application/json",
            "Accept": "application/json, text/event-stream",
        }
        async with httpx.AsyncClient(timeout=COLLIBRA_MCP_TIMEOUT_SECONDS,
auth=_collibra_basic_auth()) as client:
            response = await client.post(COLLIBRA_MCP_BASE_URL, json=payload,
headers=headers)
            response.raise_for_status()
            parsed = _parse_mcp_http_response(response.text)
        if isinstance(parsed, dict) and parsed.get("error"):
            error = parsed["error"]
            code = error.get("code") if isinstance(error, dict) else None
            message = error.get("message") if isinstance(error, dict) else str(error)
            raise ValueError(f"Collibra MCP error {code}: {message}")
        return parsed


    async def _call_collibra_mcp_tool(tool_name: str, arguments: Dict[str, Any]) -> Dict[str,
Any]:
        parsed = await _call_collibra_mcp_method(
            "tools/call",
            {"name": tool_name, "arguments": arguments},
        )
        return _normalize_collibra_tool_result(tool_name, parsed)


    def log_tool_call(
        tool_name: str,
        original_prompt: str,
        arguments: Dict[str, Any],
        tenant_id: Optional[str],
    ) -> None:
        try:
            payload = {"original_prompt": original_prompt}
            if arguments:
                payload.update(arguments)
            AgentMetadataExporter.execute_dml(
                """
                INSERT INTO raw.run_time_logs (tenant_id, tool_name, arguments, created_ts)
                VALUES (%s, %s, %s, NOW())
                """,
                (
                    str(tenant_id) if tenant_id is not None else None,
                    tool_name,
                    json.dumps(payload, default=str),
                ),
```

```
        )
    except Exception as e:
        print(f"[LOG ERROR] Failed to write log for {tool_name}: {e}")
```

8.3 Register Collibra Wrapper Tools

Add these five tool functions to the FastMCP core instance in server.py:

```
@core.tool(name="collibra_discover_data_assets")
async def collibra_discover_data_assets(original_prompt: str, *, input: str) -> Dict[str, Any]:
    """
    Query Collibra data assets using natural language.
    Args:
        original_prompt: REQUIRED. Copy the user's EXACT verbatim message
word-for-word.
        input: The question to ask the Collibra data asset discovery agent.
    """
    try:
        token = get_access_token()
        tenant_id = token.claims.get("tenant_id") if token else None
        log_tool_call("collibra_discover_data_assets", original_prompt, {"input": input},
tenant_id)
        return await _call_collibra_mcp_tool("discover_data_assets", {"input": input})
    except ValueError as ve:
        return {"error": "VALIDATION_ERROR", "details": str(ve)}
    except Exception as e:
        return {"error": "INTERNAL_ERROR", "details": str(e)}


@core.tool(name="collibra_debug_status")
async def collibra_debug_status(original_prompt: str) -> Dict[str, Any]:
    """
    Verify that Tavro can reach the Collibra MCP server and list available tools.
    Args:
        original_prompt: REQUIRED. Copy the user's EXACT verbatim message
word-for-word.
    """
    try:
        token = get_access_token()
        tenant_id = token.claims.get("tenant_id") if token else None
        log_tool_call("collibra_debug_status", original_prompt, {}, tenant_id)
        initialize = await _call_collibra_mcp_method(
            "initialize",
            {"clientInfo": {"name": "Tavro MCP Bridge", "version": "1.0"}},
        )
        tools_payload = await _call_collibra_mcp_method("tools/list")
```

```
            result_obj = initialize.get("result") if isinstance(initialize, dict) else {}
            tools_obj = tools_payload.get("result") if isinstance(tools_payload, dict) else {}
            tools = tools_obj.get("tools") if isinstance(tools_obj, dict) else []
            return {
                "status": "ok",
                "base_url": COLLIBRA_MCP_BASE_URL,
                "server": result_obj.get("serverInfo") if isinstance(result_obj, dict) else None,
                "tool_count": len(tools) if isinstance(tools, list) else 0,
                "tool_names": [t.get("name") for t in tools if isinstance(t, dict) and t.get("name")],
            }
        except ValueError as ve:
            return {"error": "VALIDATION_ERROR", "details": str(ve)}
        except Exception as e:
            return {"error": "INTERNAL_ERROR", "details": str(e)}


    @core.tool(name="collibra_get_asset_details")
    async def collibra_get_asset_details(
        original_prompt: str,
        *,
        assetId: str,
        outgoingRelationsCursor: Optional[str] = None,
        incomingRelationsCursor: Optional[str] = None,
    ) -> Dict[str, Any]:
        """
        Retrieve detailed metadata for a specific Collibra asset UUID.
        Args:
            original_prompt: REQUIRED. Copy the user's EXACT verbatim message
word-for-word.
            assetId: The UUID of the asset to retrieve.
            outgoingRelationsCursor: Optional pagination cursor for outgoing relations.
            incomingRelationsCursor: Optional pagination cursor for incoming relations.
        """
        try:
            token = get_access_token()
            tenant_id = token.claims.get("tenant_id") if token else None
            arguments: Dict[str, Any] = {"assetId": assetId}
            if outgoingRelationsCursor:
                arguments["outgoingRelationsCursor"] = outgoingRelationsCursor
            if incomingRelationsCursor:
                arguments["incomingRelationsCursor"] = incomingRelationsCursor
            log_tool_call("collibra_get_asset_details", original_prompt, arguments, tenant_id)
            return await _call_collibra_mcp_tool("get_asset_details", arguments)
        except ValueError as ve:
            return {"error": "VALIDATION_ERROR", "details": str(ve)}
        except Exception as e:
            return {"error": "INTERNAL_ERROR", "details": str(e)}
```

```
    @core.tool(name="collibra_discover_business_glossary")
    async def collibra_discover_business_glossary(original_prompt: str, *, input: str) ->
Dict[str, Any]:
        """
        Search business glossary terms and definitions in Collibra.
        Args:
            original_prompt: REQUIRED. Copy the user's EXACT verbatim message
word-for-word.
            input: The question to ask the Collibra business glossary agent.
        """
        try:
            token = get_access_token()
            tenant_id = token.claims.get("tenant_id") if token else None
            log_tool_call("collibra_discover_business_glossary", original_prompt, {"input": input},
tenant_id)
            return await _call_collibra_mcp_tool("discover_business_glossary", {"input": input})
        except ValueError as ve:
            return {"error": "VALIDATION_ERROR", "details": str(ve)}
        except Exception as e:
            return {"error": "INTERNAL_ERROR", "details": str(e)}


    @core.tool(name="create_asset")
    async def create_collibra_asset(original_prompt: str, *, payload: Dict[str, Any]) -> Dict[str,
Any]:
        """
        Create a new asset in Collibra.
        Args:
            original_prompt: REQUIRED. Copy the user's EXACT verbatim message
word-for-word.
            payload: JSON object matching Collibra's create_asset input schema.
        """
        try:
            token = get_access_token()
            tenant_id = token.claims.get("tenant_id") if token else None
            log_tool_call("create_asset", original_prompt, payload, tenant_id)
            return await _call_collibra_mcp_tool("create_asset", payload)
        except ValueError as ve:
            return {"error": "VALIDATION_ERROR", "details": str(ve)}
        except Exception as e:
            return {"error": "INTERNAL_ERROR", "details": str(e)}
```

8.4 Build and Start the Environment

Run:
docker compose up -d --build

Verify that:

- risk-mcp-server starts successfully
- collibra-mcp-server starts successfully
- No container exits unexpectedly

9. Collibra MCP Tools Exposed in Tavro

The following Collibra MCP tools were enabled:
discover_data_assets
get_asset_details
discover_business_glossary
create_asset

Tavro exposes wrapper tools for these Collibra capabilities:

collibra_discover_data_assets
collibra_get_asset_details
collibra_discover_business_glossary
create_asset

10. Tool Purpose

Tool Name	Purpose
discover_data_assets	Query available data assets using natural language.
get_asset_details	Retrieve detailed information about specific assets by UUID
discover_business_glossary	Ask questions about Collibra glossary terms and definitions
create_asset	Create a new asset in Collibra

11. Authentication

The Collibra MCP Server uses Collibra instance credentials provided through environment variables:

COLLIBRA_MCP_API_USR=<collibra_username>

COLLIBRA_MCP_API_PWD=<collibra_password>

These credentials must belong to a Collibra user with permission to access the required communities, domains, assets, and APIs.

12. Validation

The integration was validated by checking that:

- Tavro MCP server can reach the Collibra MCP Server
- Tavro can call Collibra tools through the MCP bridge
- Take a UUID and call collibra_get_asset_details with it.
- Call create_asset to create an asset.

13. Future Tool Expansion

The current implementation enables only the four Collibra MCP tools, based on the initial use cases identified for Tavro. The Collibra MCP Server supports additional capabilities that are not currently exposed through Tavro. As future requirements evolve, additional Collibra MCP tools can be enabled and mapped within Tavro.